



Applied Software Project Report

EcommerceApp: A Microservices-Based E-Commerce Platform with Event-Driven Order Processing

By

Yeswanth Kumar Gurram

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements for the degree of Master of Science in Computer Science

<September, 2026>

Scaler Mentee Email ID: yeswanthkumar.gurram@gmail.com

Thesis Supervisor: Naman Bhalla

Date of Submission: 24/09/2026

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

.....

Project Guide / Supervisor

DECLARATION

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from <Project Module start date> to <Module end date>, is the result of my own individual endeavor. The project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Yeswanth Kumar Gurram

G.yeswanth

Date: 24 Sep 2026

ACKNOWLEDGMENT

I would like to express my deepest gratitude to my family for their unwavering support and encouragement throughout this journey. A special thanks to myScaler instructors, whose guidance, mentorship, and dedication made complex concepts approachable and inspired me to push beyond my limits. I am equally thankful to all my peers, mentors, and everyone who motivated me along the way — your belief in me has been a constant source of strength. This achievement of earning my Master’s degree is not mine alone, but a shared success built on the collective support, inspiration, and kindness I received.

Table of Contents

List of Tables	7
List of Figures	8
Applied Software Project.....	9
Abstract.....	9
Project Description	10
Requirement Gathering.....	11
Functional Requirements	11
Non-Functional Requirements	12
Users and Use Cases.....	13
Feature Set.....	14
Class Diagrams.....	16
Database Schema Design.....	17
Feature Development Process.....	20
Request Flow to the Backend.....	20
Performance and Consistency Optimisation	21
Deployment Flow	23
Technologies Used	25
Java 21 and Spring Boot 3.2	25
Spring Cloud Gateway.....	25
Spring Data JPA / Hibernate	25
Spring Security and JWT (JSON Web Tokens)	25
Apache Kafka	26
MySQL and H2	26
Redis	26

Elasticsearch.....	26
Docker.....	27
Maven	27
JUnit 5 and Testcontainers.....	27
PlantUML.....	27
Conclusion	28
Key Takeaways	28
Practical Applications	28
Limitations	28
References.....	30

List of Tables

Table No.	Title	Page No.
1.1	Feature Set by Microservice	14

List of Figures

Figure No.	Title	Page No.
1.1	System Architecture Overview	11
1.2	Use Case Diagram	13
1.3	Service Class Diagram (Low-Level Design)	16
1.4	Database Schema Diagram	17
1.5	Order Placement and Fulfillment Saga (Sequence Diagram)	21

Applied Software Project

Abstract

EcommerceApp is a cloud-native, microservices-based online retail platform built to demonstrate senior-level distributed-systems engineering practices while fulfilling the functional requirements of a typical e-commerce business: user identity and profile management, product cataloging and search, shopping-cart management, order placement, payment processing, inventory reservation, and customer notifications. The system decomposes these responsibilities into eight independently deployable Spring Boot (Java 21) services -- user-service, product-service, cart-service, order-service, inventory-service, payment-service, notification-service, and an api-gateway -- coordinated through Apache Kafka using a choreographed Saga pattern that maintains data consistency across service-owned databases without a distributed transaction.

Cross-cutting concerns are addressed with JSON Web Token (JWT) based stateless authentication validated independently by every service, Redis-backed caching for the shopping cart and gateway rate-limiting, and an Elasticsearch-backed product-search index that degrades gracefully to a relational fallback query when the search cluster is unavailable. The project was developed iteratively: functional and non-functional requirements were first derived from a Product Requirements Document, translated into a Low-Level Design with class and database-schema diagrams, and finally implemented, containerised with Docker, and validated with automated unit and integration tests (JUnit 5, Testcontainers).

This report documents the requirement-gathering process, the low-level design, one exemplar feature -- the order-placement saga -- in depth, the deployment strategy, and the technologies employed, together with the architectural trade-offs made to keep the system runnable end-to-end without paid third-party infrastructure. The result is a working, testable reference implementation that mirrors real-world e-commerce platforms such as Amazon or Flipkart at a reduced but representative scale, suitable for demonstrating microservice decomposition, event-driven design, and secure API development in an academic and interview context.

Project Description

EcommerceApp reproduces, at a manageable scale, the core capabilities of a production online marketplace. It is organised as a Maven multi-module mono-repo containing one module per bounded context plus a shared 'common' library (JWT utilities, shared DTO conventions). Each service owns its own data and can, in principle, be built, tested, and deployed independently of the others -- the defining property of a microservices architecture.

The project has three objectives. First, to demonstrate microservice decomposition by business capability rather than by technical layer, so that each service maps to a single reason to change (identity, catalog, cart, order, inventory, payment, notification). Second, to demonstrate asynchronous, event-driven inter-service communication using the Saga pattern to keep data consistent across services without two-phase commit, which does not scale in a microservices setting. Third, to demonstrate the cross-cutting concerns every real system needs: stateless authentication and authorization, caching, full-text search, and API-gateway routing and rate limiting.

The relevance of this project extends beyond e-commerce: the same architectural building blocks -- a gateway in front of independently scalable services, an event bus coordinating multi-step business processes, a cache in front of a hot read path, and a search index alongside a system-of-record database -- recur in ride-hailing, food delivery, ticket booking, and logistics platforms. Figure 1.1 shows the overall system architecture, including the technology each component depends on.

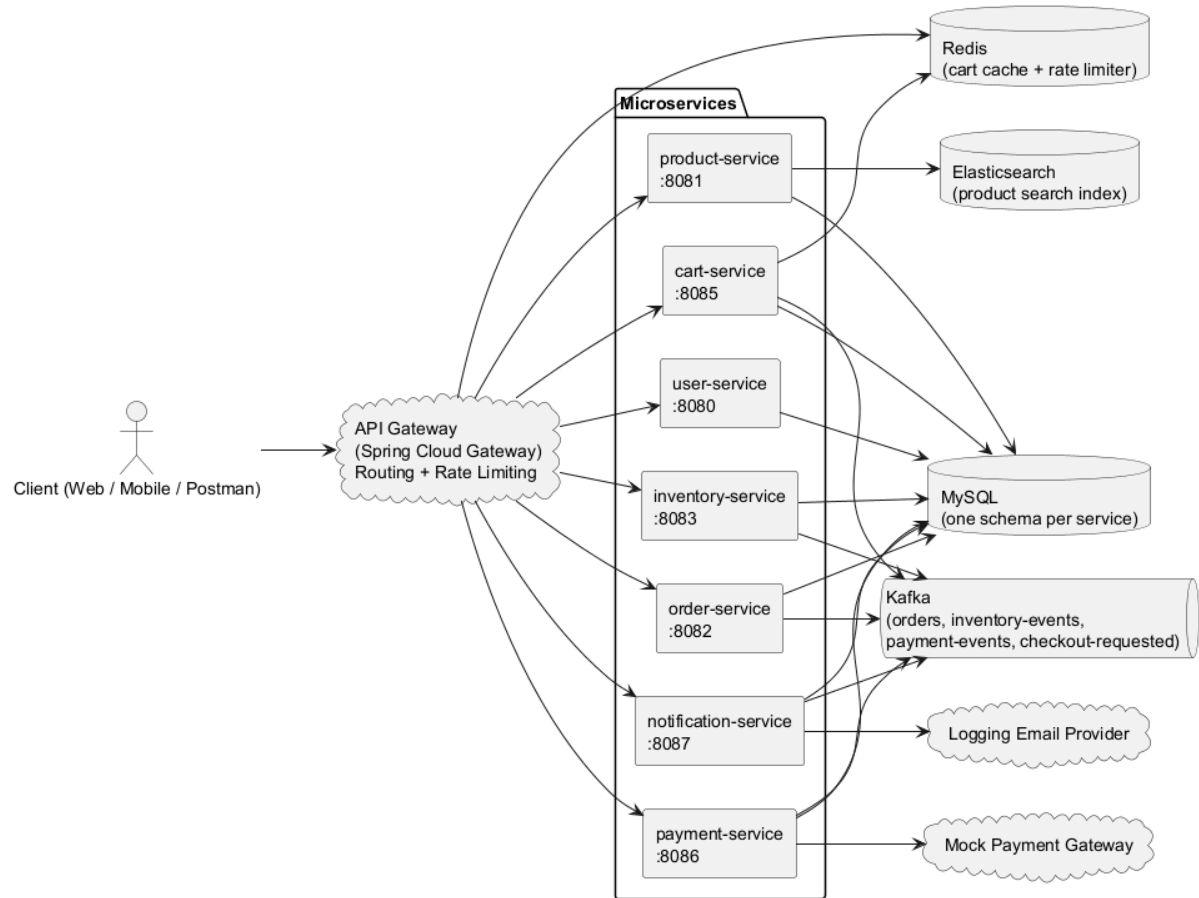


Figure 1.1: System Architecture Overview (Figure captions go below figures.)

Requirement Gathering

Functional Requirements

Functional requirements were captured in a Product Requirements Document (PRD) organised around six capability areas. The list below summarises each area and its implementation status in the current codebase; a full rationale for every deviation from the original requirement is documented in the project's 'docs/SCOPE_DEVIATIONS.md' file.

- User Management: registration (email/password and social login), login, profile view/edit, and password reset -- implemented in user-service, including JWT issuance and ownership-checked profile endpoints.

- Product Catalog: browse by category, product detail pages, and keyword search -- implemented in product-service, with Elasticsearch search and an automatic database-query fallback.
- Cart & Checkout: add to cart, review cart, and checkout with a delivery address -- implemented in cart-service, with Redis-cached reads and an asynchronous checkout event.
- Order Management: order confirmation, and order status transitions driven by the fulfillment saga -- implemented in order-service; order history/tracking endpoints are exposed per authenticated user.
- Payment: simulated multiple-payment-method authorization and success/failure outcomes -- implemented in payment-service via a swappable 'PaymentGateway' interface (a mock implementation stands in for a real processor such as Stripe).
- Authentication: secure, stateless session-less authentication using signed JWTs validated by every service -- implemented via the shared 'common' 'JwtProvider' and a per-service 'JwtFilter'.

Non-Functional Requirements

- Security: passwords are hashed with BCrypt (never stored or logged in plaintext); every write endpoint resolves the acting user from the JWT's 'userId' claim rather than a client-supplied identifier, preventing Insecure Direct Object Reference (IDOR) attacks on another user's cart, orders, or payments.
- Scalability: each service is stateless at the HTTP layer (session state lives only in the signed JWT), so any service can be horizontally scaled behind the API gateway without sticky sessions.
- Availability / graceful degradation: product search falls back to a database query if Elasticsearch is unreachable, and the shopping-cart cache falls back to the database if Redis is unreachable -- neither dependency is a hard single point of failure for its service.
- Performance: Redis cache-aside reads avoid a database round trip for the most frequently read entity (the cart); Elasticsearch provides an inverted index for

search instead of an unindexable 'LIKE' scan; the gateway's token-bucket rate limiter protects backend services from being overwhelmed by bursts of traffic.

- **Consistency:** cross-service consistency is eventual, coordinated by a choreographed Saga over Kafka, with an explicit compensating transaction (releasing a stock reservation) when a later step fails.
- **Maintainability:** bounded contexts keep each service's codebase small and focused; a shared 'common' module centralises JWT logic so it is implemented once, not copy-pasted eight times.

Users and Use Cases

The system recognises two human actors -- a Guest (unauthenticated visitor) who can browse and search the catalog, and a Registered User who can additionally manage a cart, place orders, and view order history -- plus two system actors that participate in the checkout flow: the Payment Gateway and the Notification Provider. Figure 1.2 shows the use-case diagram for the platform.

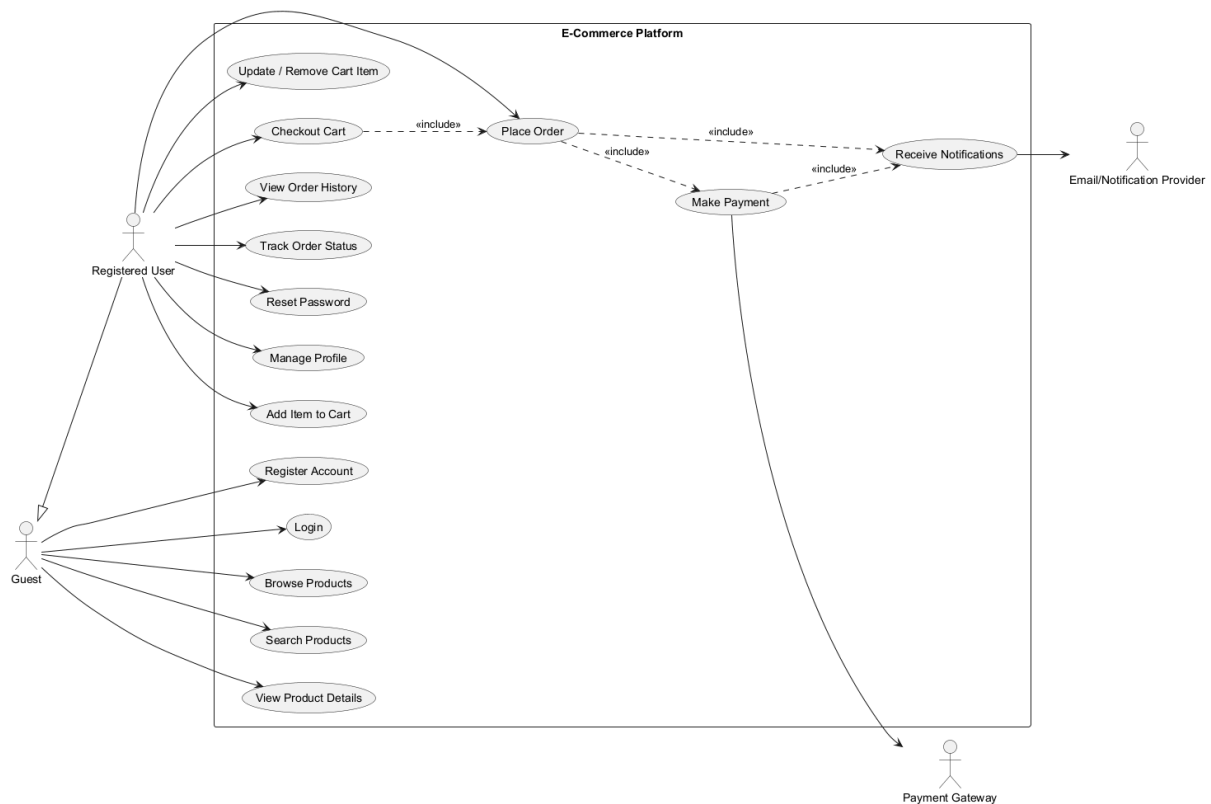


Figure 1.2: Use Case Diagram (Figure captions go below figures.)

Feature Set

Table 1.1 lists the feature set delivered by each microservice.

Table 1.1: Feature Set by Microservice (Table captions go above tables.)

#	Feature	Description	Owning Service
1	Registration & Login	Email/password registration with BCrypt hashing; JWT issuance on login; social-login endpoint	user-service
2	Profile Management	View/update own profile; ownership enforced via the JWT's email/userId claim	user-service
3	Password Reset	Token-based forgot-password / reset-password flow with a logging email provider	user-service
4	Product Catalog & Search	Category-filtered listing plus Elasticsearch keyword search with a DB LIKE fallback	product-service
5	Shopping Cart	Add / update / remove items with Redis cache-aside reads; checkout publishes a Kafka event	cart-service
6	Order Placement	Synchronous REST order placement and asynchronous cart-checkout order creation	order-service
7	Inventory Reservation	All-or-nothing stock reservation on order creation; compensating release on payment failure	inventory-service
8	Payment Processing	Event-driven authorization via a mock payment gateway; publishes success/failure events	payment-service
9	Notifications	Fan-in Kafka consumer that records/"sends" order and payment notifications	notification-service
10	Gateway Routing & Rate Limiting	Path-based routing to every service plus a Redis-backed token-bucket limiter on	api-gateway

		product browsing	
--	--	------------------	--

Class Diagrams

The Low-Level Design (LLD) translates each PRD capability into concrete classes, repositories, and their collaborations. Figure 1.3 shows the service-level class diagram spanning all eight services. Controllers depend on services, services depend on repositories (and, where relevant, an event publisher or an external client), and entities model the persisted aggregate for that bounded context -- for example, User/UserProfile/PasswordResetToken/SocialIdentity in user-service, or Order/OrderItem/DeliveryAddress in order-service.

A recurring design principle visible in the diagram is that a service never owns another service's rows: Cart/CartItem store a productId and a price snapshot rather than a foreign key into product-service's database, and Order/OrderItem treat userId and productId as external references. This is what allows each service to keep its own schema and be deployed independently -- a cross-database foreign key would couple two services' release cycles together.

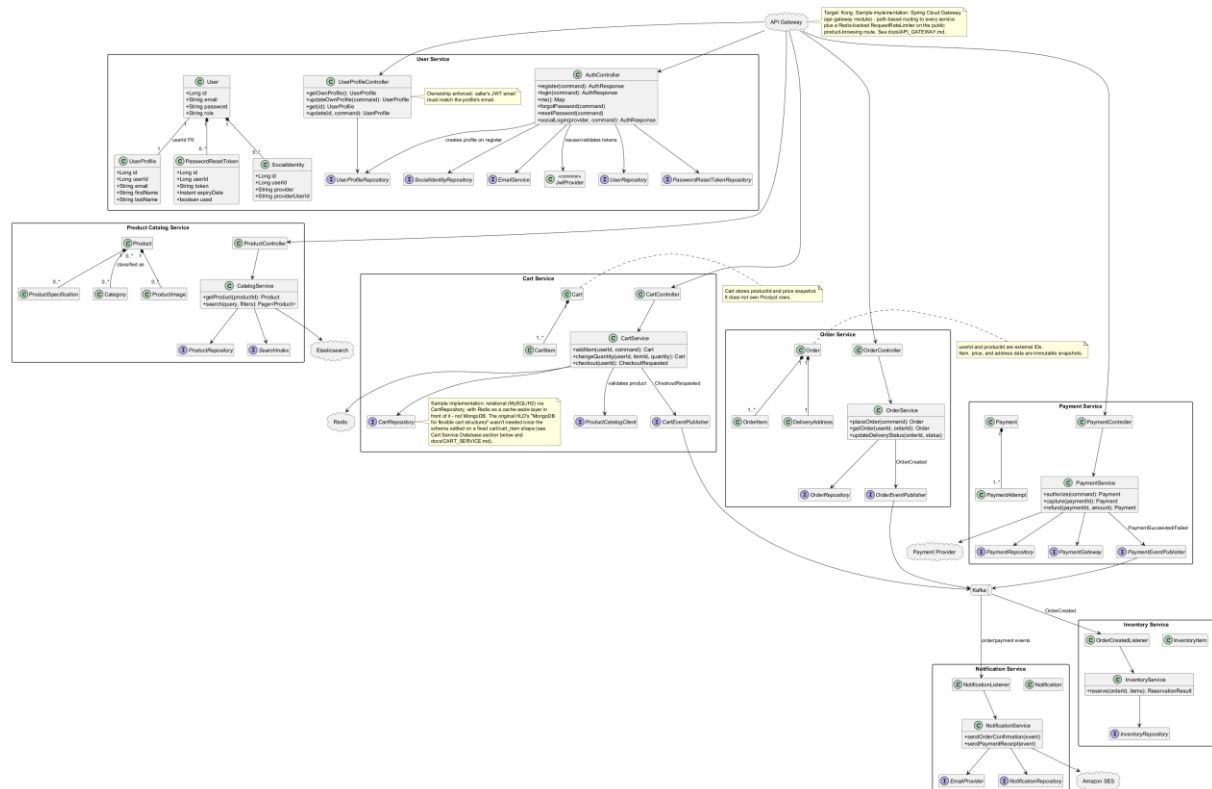


Figure 1.3: Service Class Diagram (Low-Level Design) (Figure captions go below figures.)

Database Schema Design

Each microservice owns a private relational schema; there are no cross-service foreign keys, only externally-referenced identifiers that are indexed but not constrained by the database. Monetary values use `BigDecimal/DECIMAL` types for the newer entities (`Order.totalAmount`, `Payment.amount`); the legacy `Product.price` field remains a double, a known and documented inconsistency rather than a silently-ignored one. Figure 1.4 shows the full database schema diagram across all services.

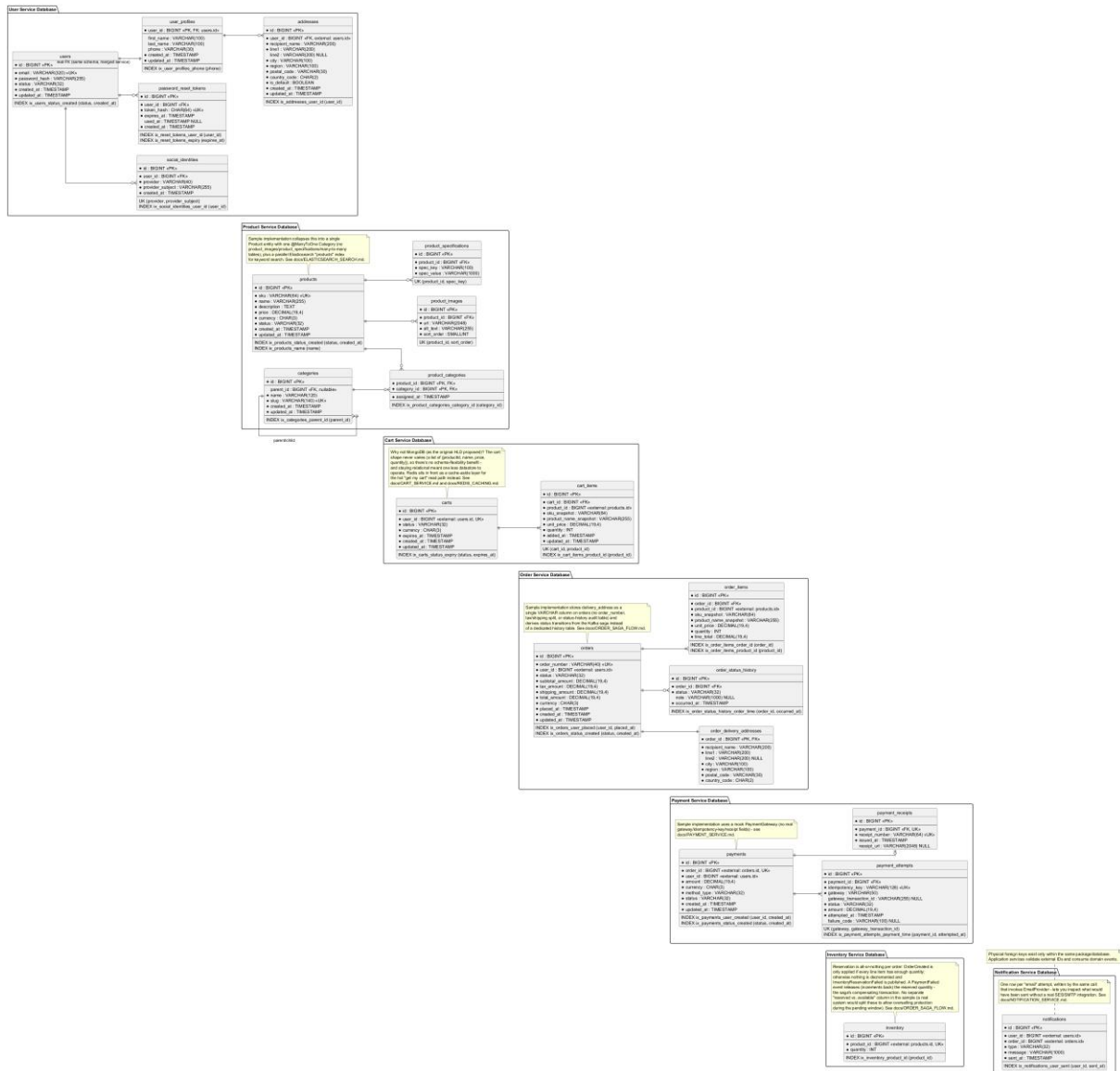


Figure 1.4: Database Schema Diagram (Figure captions go below figures.)

Sample schema design described textually:

Tables (representative columns):

- users: id (PK), email (UK), password_hash, status, created_at, updated_at
- user_profiles: user_id (PK, FK -> users.id), first_name, last_name, phone
- password_reset_tokens: id (PK), user_id (FK), token_hash, expires_at, used_at
- social_identities: id (PK), user_id (FK), provider, provider_subject
- products: id (PK), sku (UK), name, description, price, category_id (FK), status
- categories: id (PK), name, slug (UK), parent_id (FK, self-referencing)
- carts: id (PK), user_id (UK); cart_items: id (PK), cart_id (FK), product_id, product_name, unit_price, quantity
- orders: id (PK), user_id, total_amount, status, delivery_address (embedded); order_items: id (PK), order_id (FK), product_id, quantity, unit_price
- payments: id (PK), order_id, user_id, amount, status; payment_attempts: id (PK), payment_id (FK), outcome, reason, attempted_at
- inventory_items: id (PK), product_id (UK), quantity
- notifications: id (PK), user_id, channel, subject, body, sent_at

Foreign Keys and Cardinality of Relations:

- users (1) --- (1) user_profiles -- one profile per user, real FK because both live in the same schema
- users (1) --- (0..*) social_identities -- a user may link zero or more social-login providers
- users (1) --- (0..*) password_reset_tokens -- a user may request multiple reset tokens over time
- categories (1) --- (0..*) products -- a product belongs to exactly one category (many-to-one from the product's side)
- carts (1) --- (1..*) cart_items -- a non-empty cart has one or more line items
- orders (1) --- (1..*) order_items -- an order has one or more line items

- payments (1) --- (1..*) payment_attempts -- one payment may, in a production version, record multiple retried attempts

Feature Development Process

The feature chosen to illustrate the end-to-end development process in depth is order placement and fulfillment, because it is the one workflow that touches every other service in the system through the choreographed Saga pattern described in docs/ORDER_SAGA_FLOW.md.

Request Flow to the Backend

A client places an order in one of two ways: synchronously via POST /api/orders on order-service, or asynchronously by adding items to the cart and calling POST /api/cart/checkout on cart-service (which publishes a CheckoutRequested event that order-service consumes to create the order). Both paths converge on the same saga once the order row is persisted. A representative request payload for the synchronous path is:

```
{ userId: 42, items: [ { productId: 7, quantity: 2 } ], deliveryAddress: "221B Baker Street, London" }
```

The request is picked up by OrderController, which delegates to OrderService.placeOrder(...). OrderService persists the Order (status PENDING) through OrderRepository, then publishes an OrderCreated event to the 'orders' Kafka topic via OrderEventPublisher -- the same Controller -> Service -> Repository flow used throughout every service in this codebase (a thin MVC-style layering: @RestController for HTTP binding, @Service for business rules, Spring Data @Repository interfaces for persistence).

From there, the flow leaves the HTTP request/response cycle entirely and becomes event-driven, as shown in Figure 1.5: inventory-service consumes OrderCreated and attempts an all-or-nothing stock reservation, publishing InventoryReserved or InventoryReservationFailed; payment-service consumes InventoryReserved only -- it never attempts to charge a customer for stock that was not reserved -- and publishes PaymentSucceeded or PaymentFailed; order-service consumes both event streams to move the order to CONFIRMED or CANCELLED; and, if payment fails after stock was already reserved, inventory-service consumes PaymentFailed and increments the quantity back -- the saga's compensating transaction, since a Kafka publish cannot be rolled back the way a database transaction can.

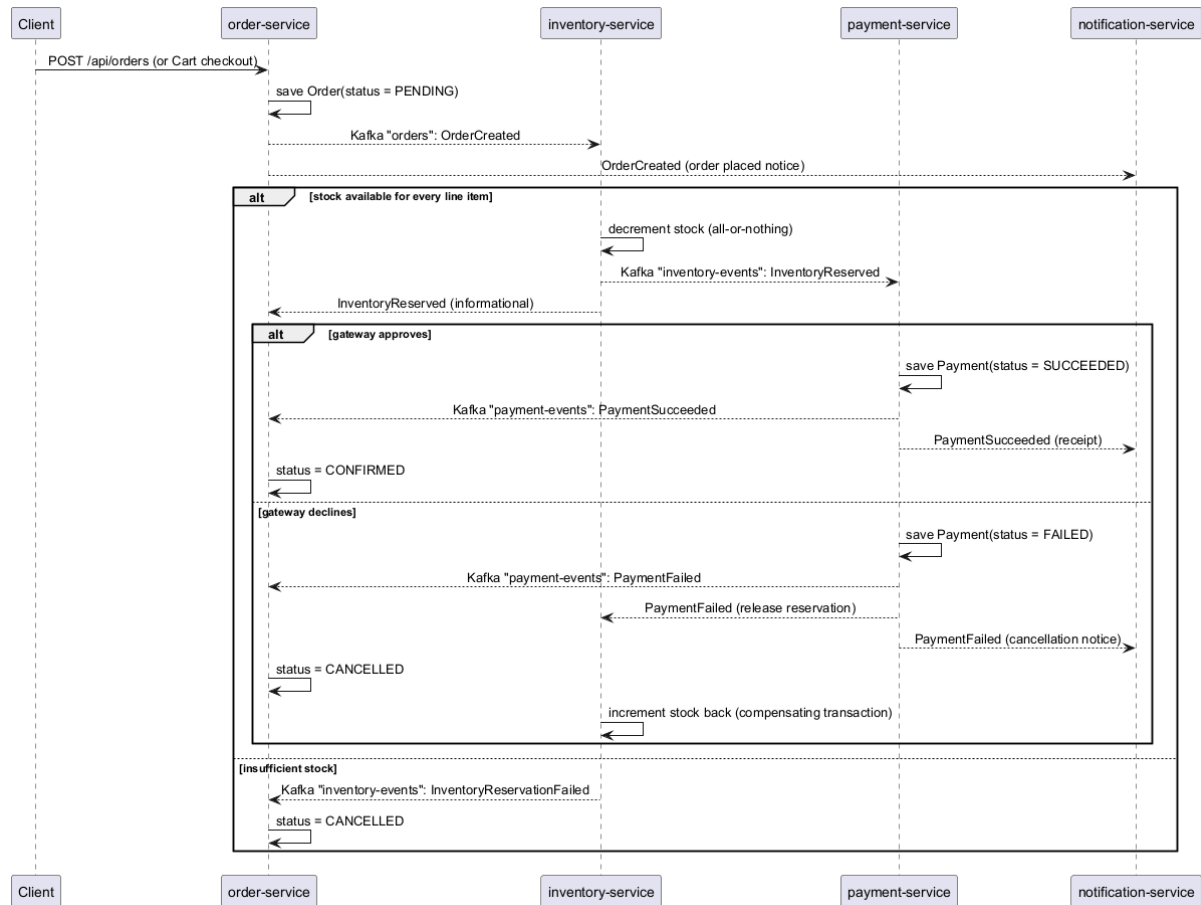


Figure 1.5: Order Placement and Fulfillment Saga (Figure captions go below figures.)

Performance and Consistency Optimisation

Two architectural choices in this feature are direct performance/consistency optimisations rather than incidental design. First, ordering payment-service to react only to InventoryReserved (rather than processing payment and inventory reservation in parallel) trades a small amount of latency for a strong data-integrity guarantee: it is architecturally impossible to charge a customer for an item that could not be reserved. Second, using Kafka as the coordination mechanism instead of a chain of synchronous REST calls between order-service, inventory-service, and payment-service means a slow or temporarily unavailable downstream service does not block the client's original request -- the client receives an immediate acknowledgment (order PENDING) while the saga completes asynchronously, which is the same UX pattern used by large retailers whose 'Order placed' confirmation appears instantly while fulfillment continues in the background.

No formal load-testing benchmark (e.g. JMeter/Gatling throughput numbers) was run against this sample, since it is an educational/interview-preparation project rather than a production deployment; the optimisations above are therefore justified architecturally in this report rather than with measured before/after figures. A natural extension of this project would be to add such benchmarks -- for example, comparing p95 latency of the cache-aside cart read against the uncached path, or comparing Elasticsearch query latency against the LIKE fallback on a catalog of realistic size.

Deployment Flow

Locally, the entire stack runs via Docker Compose (docker-compose.yml), which provisions MySQL 8.0, Zookeeper and Kafka, Redis 7, and Elasticsearch 8.11 as containers on a single Docker network; each Spring Boot module additionally ships its own Dockerfile (a two-line eclipse-temurin:21-jdk-jammy image that copies the Maven-built fat jar and runs it with java -jar), and services can be pointed at MySQL instead of their default embedded H2 database with the -Dspring.profiles.active=mysql flag.

A production deployment on AWS would replace each local container with its managed-service equivalent while keeping the same container images, as summarised below:

Component	Local (docker-compose)	Target AWS Deployment
Compute	docker-compose / individually run JARs	Each service's existing Dockerfile pushed to Amazon ECR and run on Elastic Beanstalk (Docker platform) or ECS/Fargate behind an Auto Scaling Group
Relational database	MySQL 8.0 container / H2 in-memory (tests)	Amazon RDS for MySQL, Multi-AZ, one instance (or schema) per service to preserve database-per-service ownership
Cache	Redis 7 container	Amazon ElastiCache for Redis
Search	Elasticsearch 8.11 container	Amazon OpenSearch Service
Messaging	Kafka + Zookeeper containers	Amazon MSK (Managed Streaming for Apache Kafka)
Email	LoggingEmailProvider (logs only)	Amazon SES, plugged in behind the existing EmailProvider interface
Edge / Gateway	api-gateway module (Spring Cloud Gateway) run directly	Same container behind an Application Load Balancer, inside a VPC public subnet
Secrets/config	Plaintext values in application.yml (dev only)	AWS Secrets Manager / Systems Manager Parameter Store, injected as environment variables

Networking would use a single VPC with public subnets for the Application Load Balancer (and a NAT Gateway for outbound-only private traffic) and private subnets for the services, RDS instances, ElastiCache cluster, and MSK brokers. Security Groups act as virtual, stateful firewalls at the instance/ENI level: the ALB's security group would

allow inbound HTTPS from the internet, the api-gateway service's security group would allow inbound traffic only from the ALB, each downstream service's security group would allow inbound traffic only from api-gateway (and from other services that call it directly), and each data-tier security group (RDS/ElastiCache/MSK) would allow inbound traffic only from the specific services that own that data -- following the principle of least privilege rather than one shared, permissive security group for the whole VPC.

Elastic Beanstalk (or ECS/Fargate) provides the managed-infrastructure layer requested in the PRD: it handles provisioning EC2 instances (or Fargate tasks), load-balancer registration, health checks, and rolling deployments, so that scaling a service under load is a configuration change (increase the Auto Scaling Group's desired capacity) rather than a manual operational task.

Technologies Used

The technologies below were chosen to closely mirror what a real e-commerce platform would run in production, while remaining runnable end-to-end on a single laptop for learning purposes.

Java 21 and Spring Boot 3.2

Java is a statically-typed, garbage-collected, JVM-based language widely used for backend services at scale (banking, e-commerce, telecom). Spring Boot layers convention-over-configuration, embedded servers, and auto-configuration on top of the Spring Framework, letting each microservice in this project be a small, independently runnable JAR. Real-world example: large-scale platforms such as Netflix and Amazon run substantial portions of their backend estate on the JVM for its mature tooling, garbage-collection tuning options, and enormous library ecosystem.

Spring Cloud Gateway

A reactive, non-blocking API gateway built on Spring's WebFlux stack, used here for path-based routing to every backend service and for a Redis-backed token-bucket rate limiter. Real-world example: any public-facing API (Stripe, Twitter/X, most SaaS products) fronts its services with a gateway to centralise auth, rate limiting, and routing rather than exposing every internal service directly to the internet.

Spring Data JPA / Hibernate

An Object-Relational Mapping (ORM) layer that maps Java entities to relational tables and generates SQL from repository method names or JPQL, removing most hand-written CRUD SQL. Used by every service in this project for its relational persistence. Real-world example: most enterprise Java applications with a relational database (insurance, banking, retail back-offices) use JPA/Hibernate rather than hand-rolled JDBC.

Spring Security and JWT (JSON Web Tokens)

Spring Security provides the filter-chain infrastructure for authentication/authorization; JWT issues and verifies signed, stateless JSON Web Tokens carrying a `userId` claim. Because the token itself is self-verifying, no service needs to call back to user-service on every request to check who is logged in. Real-world example: JWT-based stateless auth

is the standard approach for single-page applications and mobile clients talking to a microservice backend (used by Auth0, Firebase Auth, and most modern API platforms).

Apache Kafka

A distributed, partitioned, replicated commit log used as the message broker coordinating the order/inventory/payment/notification saga across four topics (orders, checkout-requested, inventory-events, payment-events). Kafka's consumer-group model lets multiple services independently consume the same event stream, which is what allows notification-service to be added without changing any publisher. Real-world example: Kafka underpins event pipelines at LinkedIn (where it originated), Uber (trip-event streaming), and Netflix (real-time analytics).

MySQL and H2

MySQL 8.0 is the production-profile relational database for every service (one schema per service); H2, an in-memory Java database, is the default for local development and automated tests so the whole system can run without any external database. Real-world example: MySQL (and its managed form, Amazon RDS/Aurora) is one of the most widely deployed relational databases for OLTP e-commerce and content-management workloads.

Redis

An in-memory key-value store used here for two purposes: cache-aside caching of the shopping cart in cart-service (sub-millisecond reads instead of a database round trip, with a 30-minute TTL modelling an 'abandoned cart'), and the token-bucket counters backing the gateway's rate limiter. Real-world example: Redis is near-ubiquitous as a caching layer and as the backing store for rate limiters and leaderboards in high-traffic web applications.

Elasticsearch

A distributed search and analytics engine built on Apache Lucene, used by product-service to provide relevance-ranked, typo-tolerant (fuzziness: AUTO) keyword search, with an automatic fallback to a database LIKE query if the cluster is unreachable. Real-

world example: Elasticsearch (or its OpenSearch fork) powers product search on many e-commerce sites, as well as log analytics platforms such as the ELK/Elastic Stack.

Docker

A container runtime used both to package each service as a portable image (Dockerfile per module) and to run the local development infrastructure (docker-compose.yml) for MySQL, Kafka, Redis, and Elasticsearch with a single command. Real-world example: containerisation with Docker (and orchestration with Kubernetes/ECS) is the standard packaging and deployment unit for cloud-native microservices.

Maven

The build tool managing this project's multi-module structure, dependency versions (via a shared parent POM's dependencyManagement), and packaging into runnable Spring Boot JARs. Real-world example: Maven (alongside Gradle) is one of the two dominant build tools in the Java ecosystem.

JUnit 5 and Testcontainers

JUnit 5 is the testing framework used for unit and slice tests (e.g. `@DataJpaTest`); Testcontainers spins up real, ephemeral Docker containers (MySQL, Kafka) for integration tests, so tests exercise the real database/broker rather than a hand-rolled mock. Real-world example: Testcontainers is widely used across the Java ecosystem specifically to make integration tests reproducible in CI without a shared, stateful test environment.

PlantUML

A text-to-diagram tool used to author and render this report's class, database-schema, use-case, architecture, and sequence diagrams from version-controlled .puml source files rather than hand-drawn images, so the diagrams can be regenerated whenever the design changes. Real-world example: PlantUML (and similar diagram-as-code tools such as Mermaid) is commonly used to keep architecture diagrams in sync with a codebase over time.

Conclusion

Key Takeaways

Building EcommerceApp end-to-end reinforced several concepts that are difficult to internalise from reading alone: why the Saga pattern (choreography specifically) is the practical alternative to a distributed two-phase commit across independently owned databases; how a compensating transaction differs from a database rollback; why stateless JWT authentication scales better than server-side sessions across many independently deployed services; the cache-aside pattern and its consistency trade-offs versus write-through caching; and why a search engine like Elasticsearch is architecturally a secondary, denormalised read model (CQRS in miniature) rather than a replacement for the system-of-record database.

Practical Applications

Every pattern demonstrated in this project generalises well beyond e-commerce. Event-driven sagas apply to any multi-step business process spanning services owned by different teams (travel booking: flight + hotel + car; food delivery: order + restaurant-acceptance + rider-assignment + payment). API-gateway routing and rate limiting apply to any system exposing multiple backend services through one public API. Cache-aside and search-index-alongside-database apply to any read-heavy workload with a hot key or free-text-search requirement.

Limitations

This sample intentionally trades production-completeness for learning clarity, and the limitations are documented explicitly (see docs/SCOPE_DEVIATIONS.md) rather than silently omitted. Kafka consumers in this project are not idempotent, so an at-least-once redelivery of OrderCreated would double-reserve stock; a production system would need a deduplication/idempotency-key strategy. Payment processing uses a deterministic mock gateway rather than a PCI-DSS-scoped real processor, and there are no refunds, receipts, or retry/idempotency keys on payment attempts. Product-to-search synchronisation is an inline dual write on create only (no update/delete sync, no CDC pipeline), so the search index can drift from the database over time. There is no service-discovery layer (services locate each other via hardcoded host:port configuration) and no gateway-level JWT

validation (every service re-validates its own token). Running the full stack -- nine application services plus four infrastructure containers -- also has real infrastructure-cost implications in a cloud deployment; a production rollout would likely start by consolidating the lowest-traffic services (e.g. notification-service, inventory-service) onto shared compute, or moving to serverless containers (AWS Fargate) to avoid paying for idle capacity, before scaling out further as traffic grows.

References

Spring Boot Reference Documentation. Spring.io. Retrieved September 2026, from <https://spring.io/projects/spring-boot>

Apache Kafka Documentation. Apache Software Foundation. Retrieved September 2026, from <https://kafka.apache.org/documentation/>

Redis Documentation. Redis Ltd. Retrieved September 2026, from <https://redis.io/docs/latest/>

Elasticsearch Reference Guide. Elastic N.V. Retrieved September 2026, from <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>

Introduction to JSON Web Tokens. JWT.io / Auth0. Retrieved September 2026, from <https://jwt.io/introduction>

Richardson, C. Pattern: Saga. Microservices.io. Retrieved September 2026, from <https://microservices.io/patterns/data/saga.html>

Fowler, M. (2014). Microservices. martinowler.com. Retrieved September 2026, from <https://martinfowler.com/articles/microservices.html>

Docker Documentation. Docker Inc. Retrieved September 2026, from <https://docs.docker.com/>